PES University
Department of Computer Science and Engineering
UE23CS352B — Object Oriented Analysis & Design

Online Food Ordering System

OOAD Mini Project Report

Member	Name	USN	Feature
1	Aarna M	PES2UG23CS009	Auth & User Management
2	Adarsh L	PES2UG23CS025	Restaurant & Menu Browsing
3	Ahana M	PES2UG23CS035	Cart & Order Placement
4	Akshaya Lakshmi Narasimhan	PES2UG23CS046	Payment, Tracking & Reviews

Semester 6 'A' | Academic Year 2025–26

Stack: Spring Boot 3.2 + Thymeleaf + H2 + Spring Security + Spring Data JPA + Lombok

1. Problem Statement

The Online Food Ordering System is a web application that streamlines the process of browsing restaurants, ordering food, and managing deliveries. The system serves three types of users: Customers, who browse restaurants, manage their cart, place orders, and track deliveries; Admins, who manage restaurants, menus, and orders; and Drivers, who are tracked through the delivery pipeline.

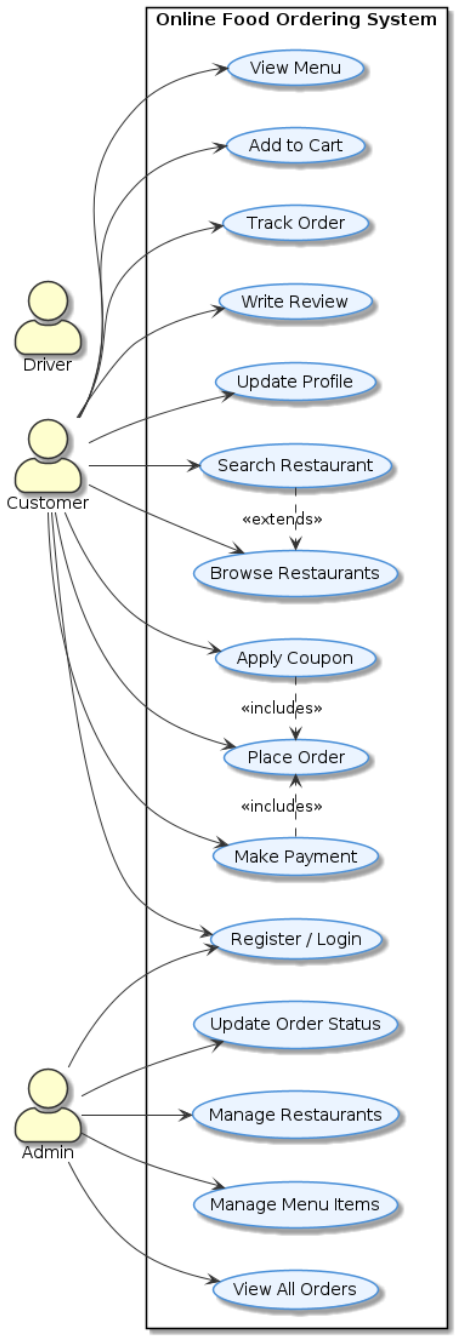
The primary goals are: (1) provide a seamless end-to-end food ordering experience from restaurant discovery to delivery tracking; (2) demonstrate proper application of MVC architecture, design patterns, and SOLID principles using the Spring Boot framework; and (3) implement a secure, role-based access system ensuring customers and admins have appropriately scoped access.

Technology Stack

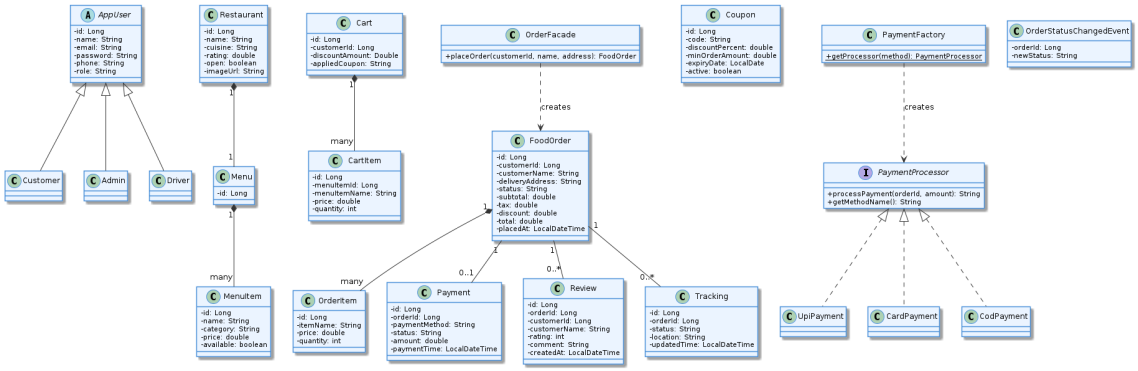
Component	Technology
Backend Framework	Spring Boot 3.2.5
View Layer	Thymeleaf 3 + Bootstrap 5
Database	H2 In-Memory (JPA/Hibernate)
Security	Spring Security 6 (BCrypt + Role-based)
ORM	Spring Data JPA + Lombok
Build Tool	Maven 3.9

2. UML Models

2.1 Use Case Diagram

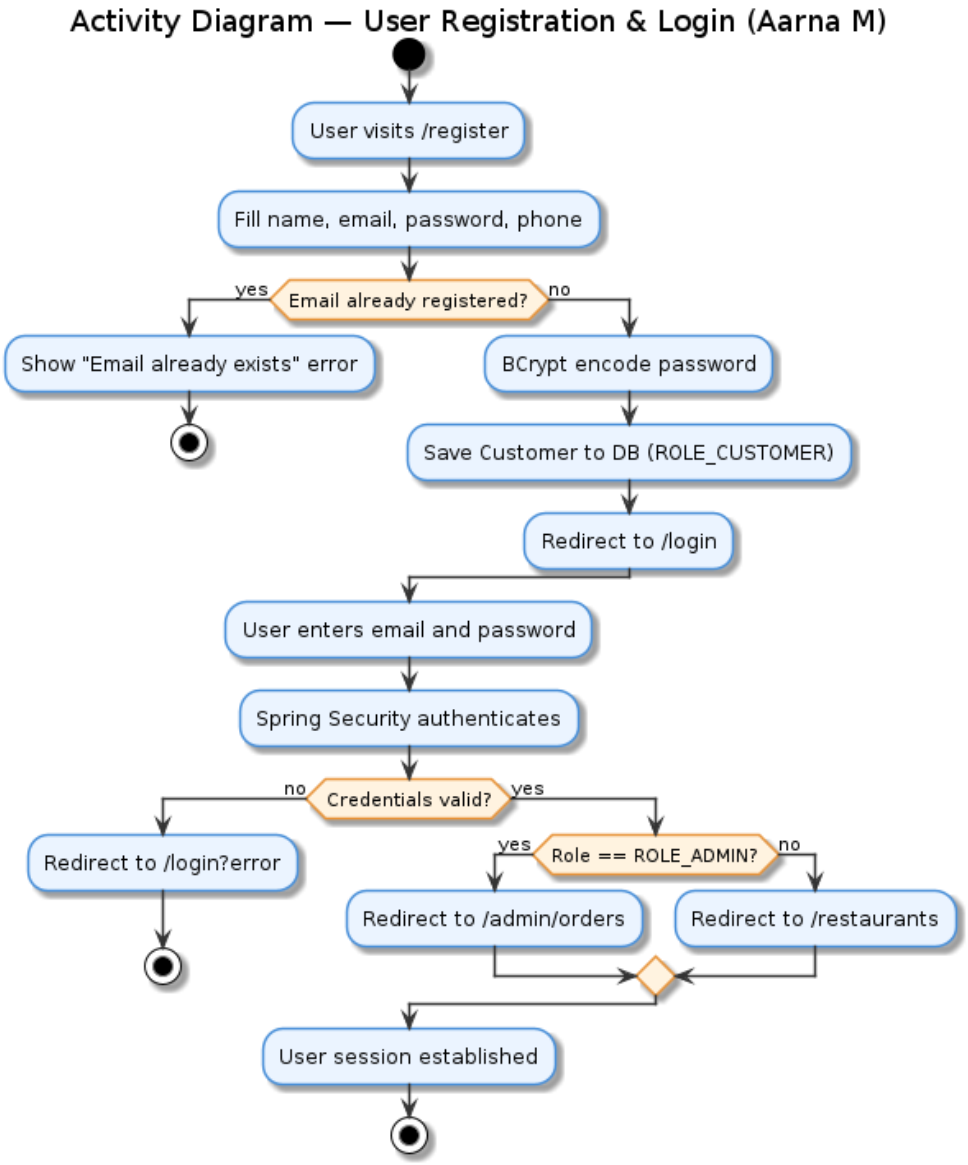


2.2 Class Diagram



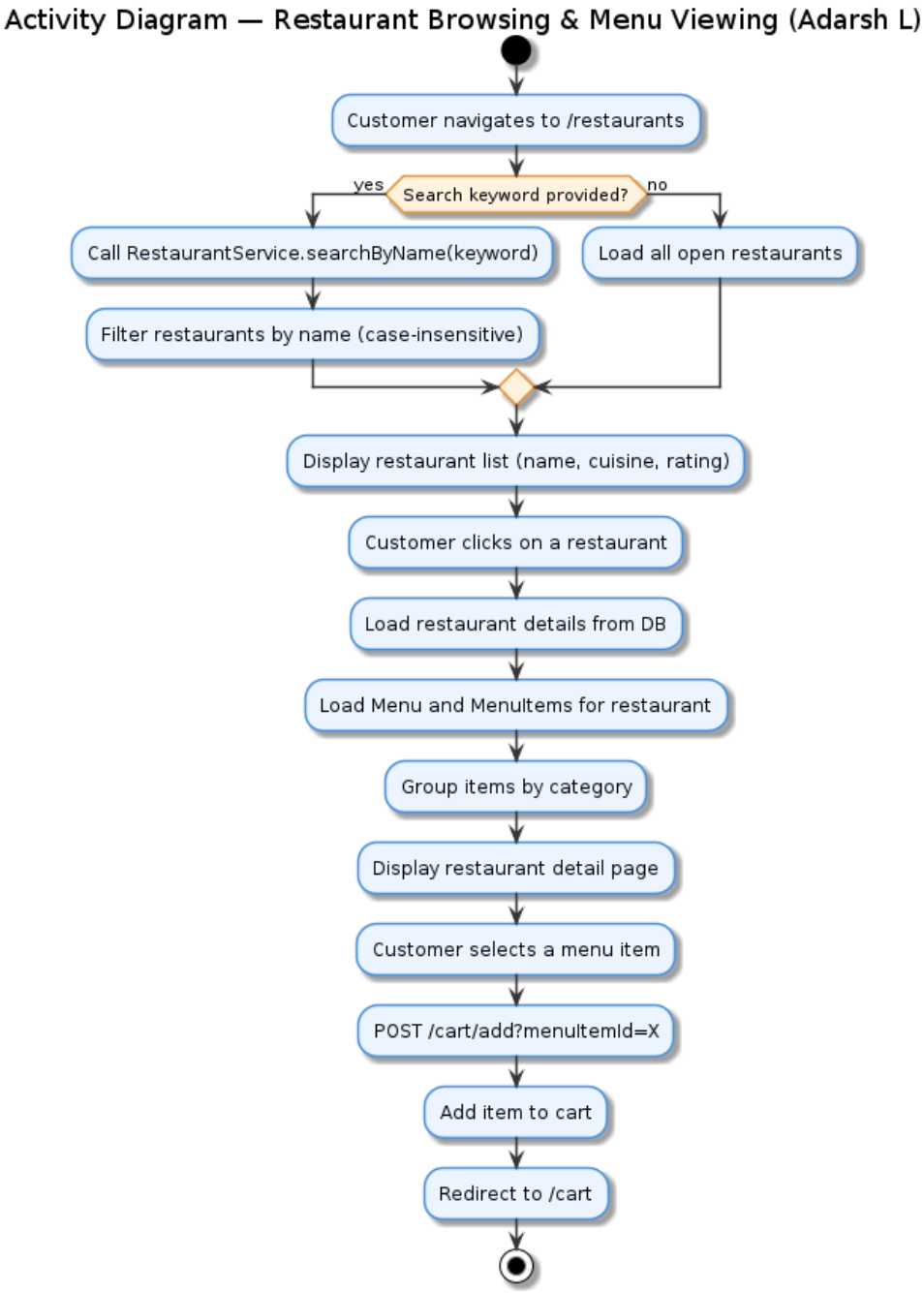
2.3 Activity Diagrams

Aarna M — User Registration & Login



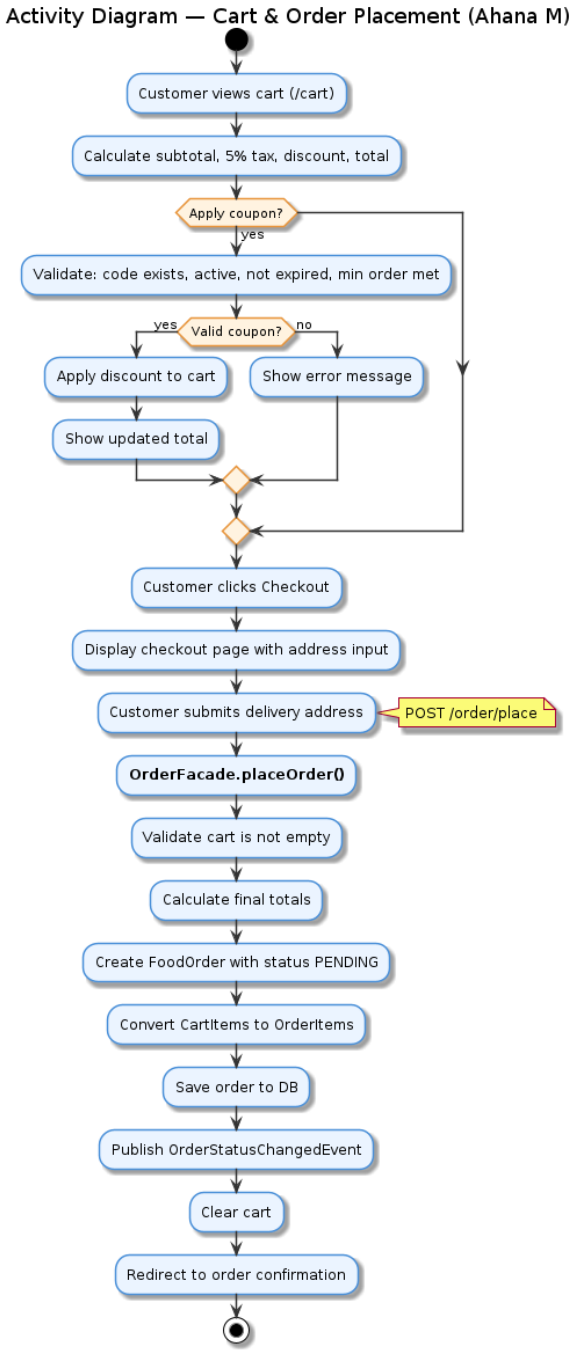
Aarna M — User Registration & Login

Adarsh L — Restaurant Browsing & Menu Viewing



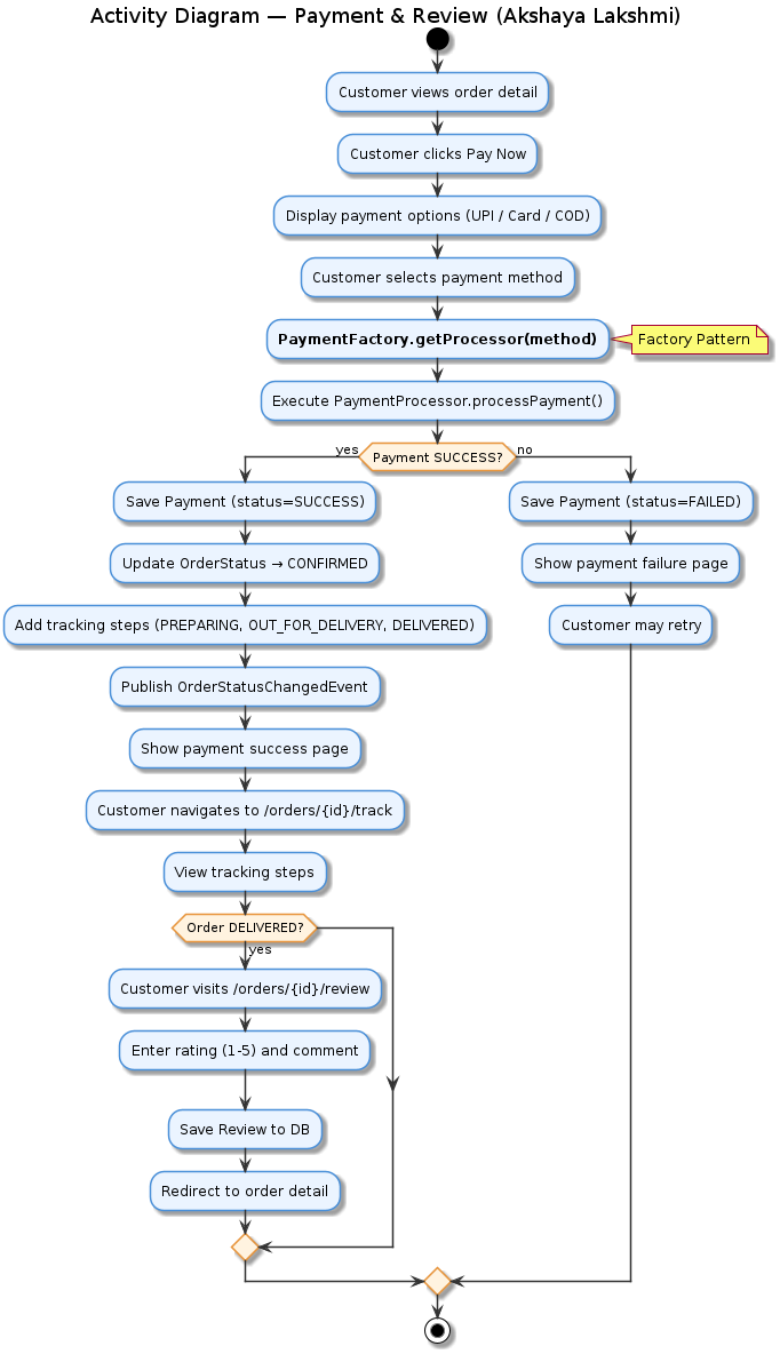
Adarsh L — Restaurant Browsing & Menu Viewing

Ahana M — Cart Management & Order Placement



Ahana M — Cart Management & Order Placement

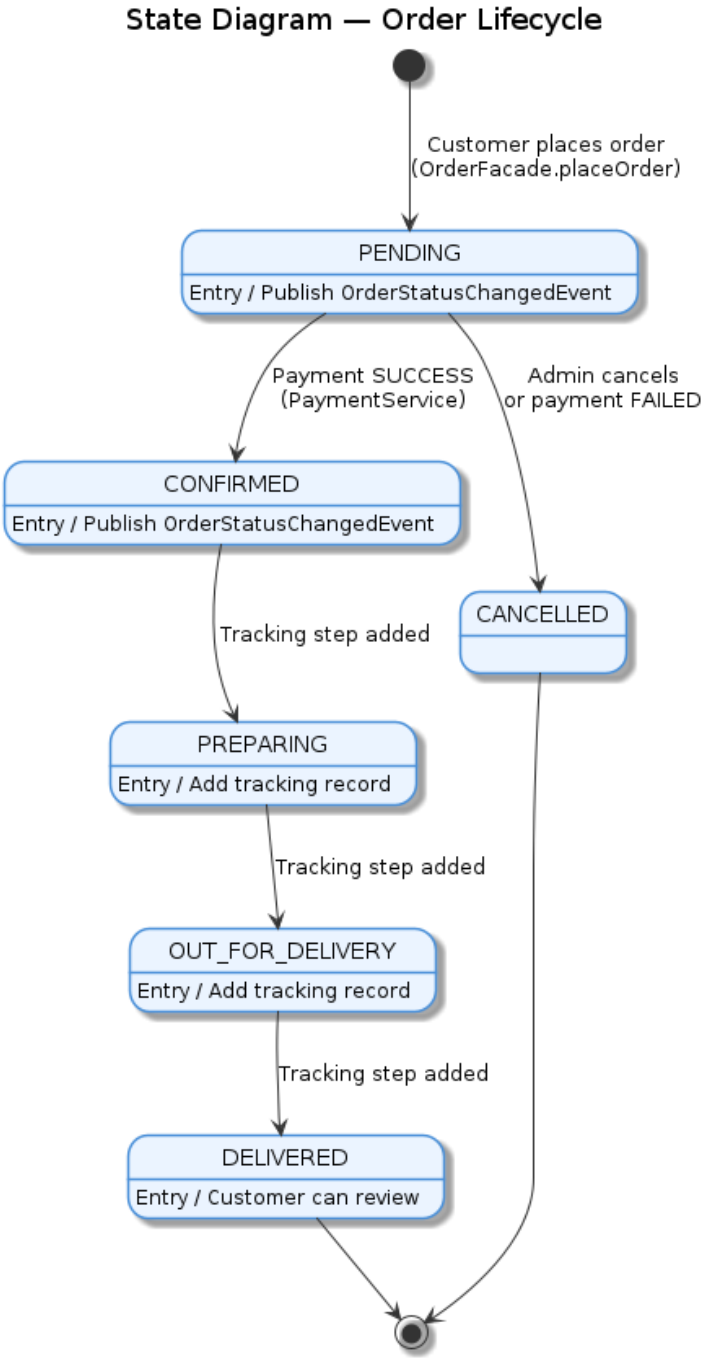
Akshaya Lakshmi — Payment Processing & Review



Akshaya Lakshmi — Payment Processing & Review

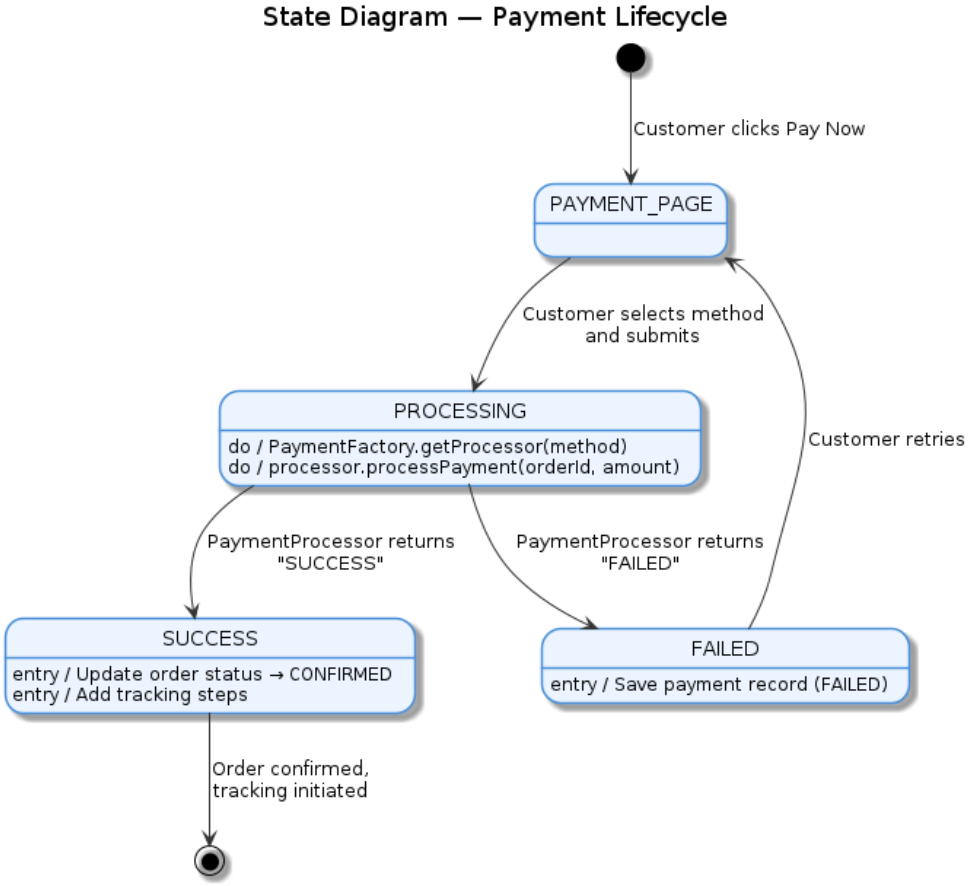
2.4 State Diagrams

Order Lifecycle State Diagram



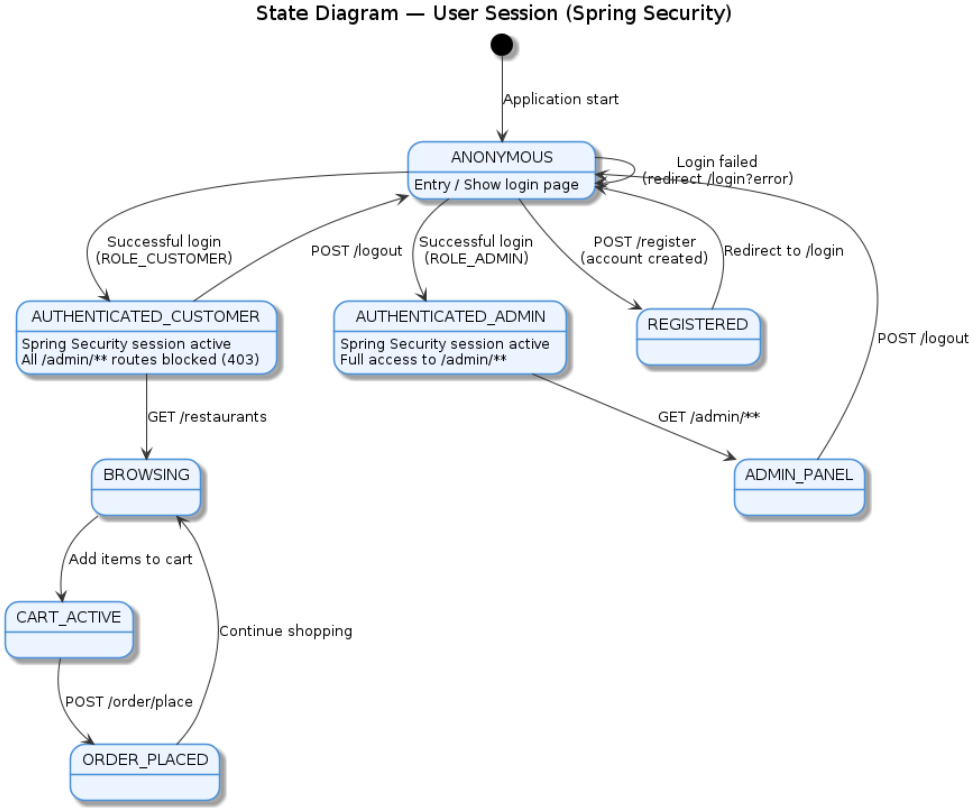
Order Lifecycle State Diagram

Payment Lifecycle State Diagram



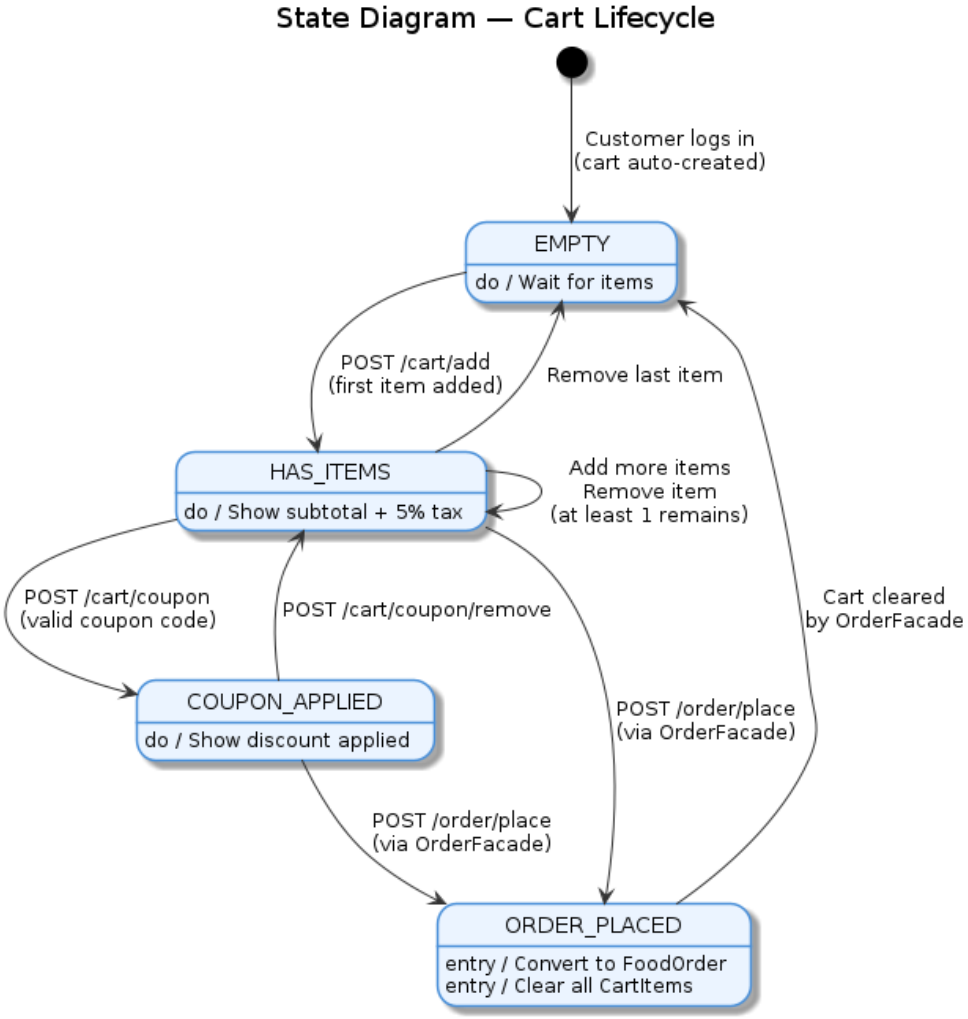
Payment Lifecycle State Diagram

User Session State Diagram



User Session State Diagram

Cart Lifecycle State Diagram



Cart Lifecycle State Diagram

3. Architecture Pattern — MVC

The application follows the Model-View-Controller (MVC) architecture enforced by Spring Boot. Each layer has a single, well-defined responsibility:

Layer	Technology	Responsibility
Model	JPA Entities + Repositories	Data representation and DB access
View	Thymeleaf HTML Templates	Rendering UI — no business logic
Controller	Spring @Controller classes	Route HTTP requests to services
Service	Spring @Service classes	All business logic lives here

Example: In CartController.viewCart(), the controller retrieves the logged-in user via Spring Security's @AuthenticationPrincipal, calls CartService to fetch cart data and compute totals, then passes the results to the Thymeleaf template. Zero business logic lives in the controller.

4. Design Patterns

4.1 Factory Pattern (Creational) — Akshaya Lakshmi Narasimhan

The PaymentFactory class creates the appropriate PaymentProcessor implementation based on the selected payment method. This decouples the payment selection logic from the payment execution logic, and follows the Open/Closed Principle — adding a new payment method (e.g., WalletPayment) requires only a new class and one new case in the factory.

```
PaymentFactory.getProcessor("UPI") → UpiPayment PaymentFactory.getProcessor("CARD")
→ CardPayment PaymentFactory.getProcessor("COD") → CodPayment
```

4.2 Facade Pattern (Structural) — Ahana M

OrderFacade.placeOrder() provides a single entry point for the entire order placement pipeline, hiding five subsystem interactions from the controller: (1) CartService validates the cart, (2) CartService calculates totals, (3) OrderRepository persists the order, (4) ApplicationEventPublisher fires the Observer event, and (5) CartService clears the cart. The controller calls one method; the complexity is entirely hidden.

```
orderFacade.placeOrder(userId, name, address) // one call in controller
```

4.3 Observer Pattern (Behavioral) — Ahana M + Akshaya Lakshmi

Spring's ApplicationEventPublisher fires an OrderStatusChangedEvent whenever an order's status changes. OrderStatusListener listens for this event and logs the status change. This decouples the order status logic from any notification or logging side effects — new listeners can be added without modifying the publisher.

```
eventPublisher.publishEvent(new OrderStatusChangedEvent(this, orderId, status)); //
Listener automatically invoked by Spring — zero coupling
```

4.4 MVC Pattern (Framework-enforced) — All Members

Spring MVC enforces separation of concerns across the entire codebase. Controllers route requests, Services handle business logic, Repositories abstract data access, and Thymeleaf templates render the view. This counts as the fourth required pattern per project guidelines.

5. Design Principles

Principle	How Applied in This Project
Single Responsibility (SRP)	Each class has exactly one job. CartService only manages cart and coupon logic. Orde
Open/Closed (OCP)	PaymentProcessor interface is open for extension (add WalletPayment class) without m
Dependency Inversion (DIP)	Controllers and services depend on abstractions, not implementations. Spring @Autow
DRY (Don't Repeat Yourself)	calculateSubtotal(), calculateTax(), calculateTotal() are defined once in CartService. Pr

6. Individual Contributions

Aarna M — PES2UG23CS009

Feature: Auth & User Management

• User entity hierarchy (AppUser, Customer, Admin, Driver) with SINGLE_TABLE inheritance

• Spring Security configuration — BCrypt password encoding, form login, role-based access (/admin/** → ROLE_ADMIN)

• UserDetailsService backed by UserRepository for Spring Security authentication

• AuthController: /register (GET+POST), /login (GET) — Spring Security handles POST

• ProfileController: /profile (GET), /profile/edit (POST) using @AuthenticationPrincipal

• UserService: register(), updateProfile(), findByEmail()

• login.html, register.html, profile.html templates

Adarsh L — PES2UG23CS025

Feature: Restaurant & Menu Browsing

• Restaurant, Menu, MenuItem JPA entities with @OneToOne and @OneToMany relationships

• RestaurantController: /restaurants (with ?search= query param), /restaurants/{id}

• RestaurantService: getAllRestaurants(), searchByName(), getById(), addRestaurant(), addMenuItem()

• Admin CRUD for restaurants and menu items (consolidated in AdminController)

• restaurants.html, restaurant-detail.html, admin-restaurants.html, admin-menu-items.html

• Seed data: 4 restaurants, 13 menu items across 3 menus

Ahana M — PES2UG23CS035

Feature: Cart & Order Placement

• Cart, CartItem, FoodOrder, OrderItem, Coupon JPA entities

• CartService: addItem(), removeItem(), applyCoupon(), calculateSubtotal/Tax/Total() (DRY methods)

• CartController: /cart, /cart/add, /cart/remove/{id}, /cart/coupon, /checkout

• OrderFacade (Facade Pattern): orchestrates 5 subsystems in placeOrder()

• OrderService: getById(), getOrdersByCustomer(), updateStatus(), filterByStatus()

• OrderController: /order/place, /orders, /orders/{id}, /orders/{id}/confirm

• OrderStatusChangedEvent + OrderStatusListener (Observer Pattern — shared with Akshaya)

• cart.html, checkout.html, order-confirmation.html templates

Akshaya Lakshmi Narasimhan — PES2UG23CS046

Feature: Payment, Tracking & Reviews

• Payment, Review, Tracking JPA entities

• PaymentProcessor interface + UpiPayment, CardPayment, CodPayment implementations

• PaymentFactory (Factory Pattern): creates correct processor by method string

• PaymentService: processPayment(), adds tracking steps on success

• ReviewService: submitReview()

• TrackingService + TrackingController: /orders/{id}/track

• AdminController: /admin/orders (list + filter by status), /admin/orders/{id}/status (update)

• orders.html, order-detail.html, payment.html, payment-success.html, payment-failure.html, review.html, tracking.html, admin-orders.html

7. GitHub Repository

The complete source code is available at the team's public GitHub repository. The repository contains the full Spring Boot project, all UML diagrams, and this report.

Repository URL: https://github.com/team11-pesu/online-food-ordering-system

(Replace with actual public repository URL before submission)

Demo Credentials:
Customer login: alice@demo.com / pass123
Admin login: admin@demo.com / admin

Running the Project

```
cd team11 ./mvnw spring-boot:run # Visit: http://localhost:8080
```